26T1 Week01

Welcome to COMP2511!

Software Design & Architecture

Annelise

intro

Welcome to COMP2511!

This course will be a bit different to what you've done before. We'll focus more on design techniques – what makes **software flexible, reusable and maintainable –** as well as the basics of **software architecture**.

Get keen!

course info

- Weekly lab exercises
- Lab attendance is not strictly mandatory, but you'll need to get your lab exercises marked off by a tutor in person
- Lab exercises can only be marked within 2 weeks of the due date. i.e. Week 2 work must be marked by/during Week 4 lab.
- Email me with any questions! annelise.koh@student.unsw.edu.au

course info

- Coursework: 15%
  - Eight labs total, each worth ten marks. We will add all eight lab marks and **cap total coursework marks at 70** (which will translate to 15%), leaving one lab's worth of marks as a **buffer**.
- Assignments: 35%
- Final Exam: 50%
  - Tutorial and lab content will make up at least 50% of the exam !!1!
- More info on the course outline

- Name
- Year
- Degree
- Something you're good at AND/OR
- Something you're bad at

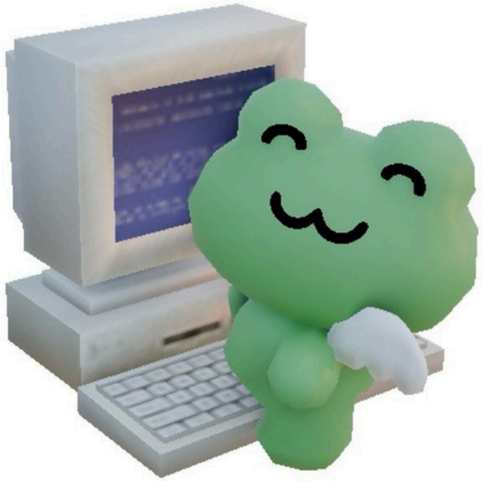
introductions

UNSW has decided that they want to create their own light rail, which takes students from upper campus to lower campus.

Design a solution for this - how will it work?

What will need to be changed about the campus layout for it to work?

design problem



demo time!

classes

A blueprint that defines data and behaviour for objects.

```
public class MyClass {
  ...
}
```

- All code that can run or do something **must be within a class**.
- Similar to structs from C, but way more extensive
- This is because Java is an **object-oriented programming language**.
- Classes can do a lot more than just hold code, but you'll learn more about them throughout the term!
- Even though there is boilerplate, you'll need a class for almost everything 😅

main function

- This is how we write the main function in Java; i.e. the code's starting point
- You can see that it's quite similar to C, you just need to **wrap a class around it**.
- Anything in this function **will be run when you click "run"** using the VSCode tool.
- You can use VSCode's autocomplete to write it.

```c
// main function in C
int main(int argc, char *argv[]) {
  // run your code here!
}
```

★ ★ ★

```java
public class MyClass {
  public static void main(String[] args) {
    // run your code here!
  }
}
```

properties

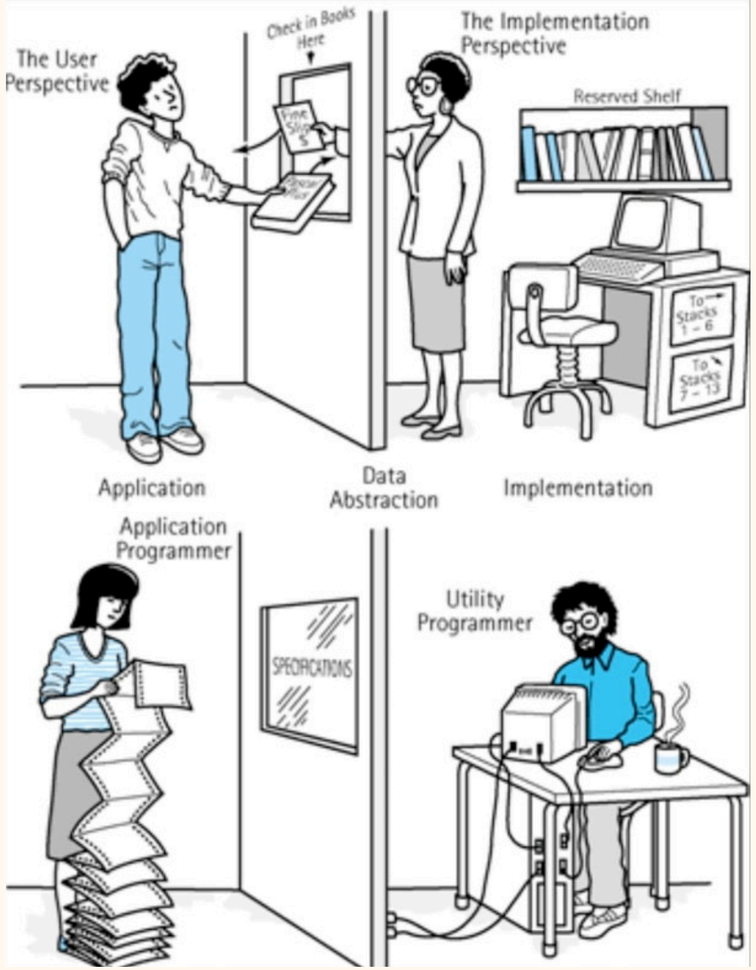
```java
public class Person {
  private String name;
  private int age;
  public Person(String name, int age) {
    this.name = name;
    this.age = age;
  }

  public String getName() {
    return this.name;
  }

  public void setAge(int age) {
    this.age = age;
  }
}
```

- In Java, **classes can have properties, and methods** associated with them.
- To initialise classes, we have to declare a **constructor**.
  - Think of it as the first “function” that runs when you create a class.
  - It can set class properties, which is data stored inside a class (like a struct from C).
- You can write other methods to use the data inside your class.
- In this case, we can get the name, and set the age.

abstraction



The process of creating a Class, assigning it properties and using methods to play with them is an OOP concept called **abstraction**!

- Abstraction **hides implementation** details and **exposes only the essential features** of an object for usage.
- In our case, we only cared about printing the Shouted string. We hid the logic and storage away.
- From the POV of a user of the class, we just had to put the contents in and get out a result

lab time!

SitarME306